

符号扩展 零标志 ZF 溢出标志 OF 符号标志 SF 进 / 借位标志 CF

2. 简单回答下列问题。

- (1) 为什么计算机内部采用二进制表示信息？既然计算机内部所有信息都用二进制表示，为什么还要用到十六进制或八进制数？
- (2) 常用的定点数编码方式有哪几种？通常它们各自用来表示什么信息？
- (3) 为什么现代计算机中大多用补码表示带符号整数？
- (4) 在浮点数的基数和总位数一定的情况下，浮点数的表示范围和精度分别由什么决定？两者如何相互制约？
- (5) 为什么要对浮点数进行规格化？有哪两种规格化操作？
- (6) 为什么计算机处理汉字时会涉及不同的编码（如输入码、内码、字模码）？说明这些编码中哪些用二进制编码，哪些不用二进制编码，为什么？

3. 实现下列各数的转换。

- (1) $(28.125)_{10} = (\quad)_2 = (\quad)_8 = (\quad)_{16}$
- (2) $(1001111.01)_2 = (\quad)_{10} = (\quad)_8 = (\quad)_{16}$
- (3) $(8A.B)_{16} = (\quad)_{10} = (\quad)_2$

4. 假定机器数为 8 位（1 位符号，7 位数值），写出下列各二进制数的原码表示。

+0.10011, -0.10011, +1.0, -1.0, +0.01101, -0.01101, +0, -0

5. 假定机器数为 8 位（1 位符号，7 位数值），写出下列各二进制数的补码和移码表示。

+10011, -10011, +1, -1, +1101, -1101, +0, -0

6. 已知 $[x]_{\text{补}}$ ，求 x

- (1) $[x]_{\text{补}} = 1100\ 0101$ (2) $[x]_{\text{补}} = 1000\ 0000$ (3) $[x]_{\text{补}} = 0111\ 0010$ (4) $[x]_{\text{补}} = 1111\ 0010$

7. 某 32 位字长的机器中带符号整数用补码表示，浮点数用 IEEE 754 标准表示，寄存器 R1 和 R2 的内容分别为 0000 108BH 和 8080 108BH。不同指令对寄存器进行不同的操作，因而不同指令执行时寄存器内容对应的真值不同。假定执行下列运算指令时，操作数为寄存器 R1 和 R2 的内容，则 R1 和 R2 中操作数的真值分别为多少？

- (1) 无符号整数加法指令
- (2) 带符号整数乘法指令
- (3) 单精度浮点数减法指令

⑧ 假定机器 M 的字长为 32 位，用补码表示带符号整数。表 2.12 中第一列给出了在机器 M 上执行的 C 程序中的关系表达式，请参照已有的表栏内容完成表中后三栏内容的填写。

表 2.12 题 8 表

关系表达式	运算类型	结果	说明
$0 = 0U$	无符号整数 带符号整数	0 1	$11\cdots 1B\ (2^{32}-1) > 00\cdots 0B(0)$ $011\cdots 1B\ (2^{31}-1) > 100\cdots 0B\ (-2^{31})$
$-1 < 0$			
$-1 < 0U$			
$2147483647 > -2147483647 - 1$			
$2147483647U > -2147483647 - 1$			
$2147483647 > (\text{int})\ 2147483648U$			
$-1 > -2$			
$(\text{unsigned})\ -1 > -2$			

16. 对于一个 n ($n \geq 8$) 位的变量 x , 请根据 C 语言中按位运算的定义, 写出满足下列要求的 C 语言表达式。

- (1) x 的最高有效字节不变, 其余各位全变为 0。
- (2) x 的最低有效字节不变, 其余各位全变为 0。
- (3) x 的最低有效字节全变为 0, 其余各位取反。
- (4) x 的最低有效字节全变 1, 其余各位不变。

17. 以下是一个由反汇编器生成的一段针对某小端方式处理器的机器级代码表示文本, 其中, 最左边是指令所在的存储单元地址, 冒号后是指令的机器码, 最右边是对应汇编指令。已知反汇编输出的机器数用补码表示, 给出指令代码中画线部分机器数对应的真值。

```
80483d2: 81 ec b8 01 00 00
80483d8: 8b 55 08
80483db: 83 c2 14
80483de: 8b 85 58 fe ff ff
80483e4: 03 02
80483e6: 89 85 74 fe ff ff
80483ec: 8b 55 08
80483ef: 83 c2 44
80483f2: 8b 85 c8 fe ff ff
80483f8: 89 02
80483fa: 8b 45 10
80483fd: 03 45 0c
8048400: 89 85 ec fe ff ff
8048406: 8b 45 08
8048409: 83 c0 20
```

```
sub    $0x1b8, %esp
mov    0x8(%ebp), %edx
add    $0x14, %edx
mov    0xfffffe58(%ebp), %eax
add    (%edx), %eax
mov    %eax, 0xfffffe74(%ebp)
mov    0x8(%ebp), %edx
add    $0x44, %edx
mov    0xfffffec8(%ebp), %eax
mov    %eax, (%edx)
mov    0x10(%ebp), %eax
add    0xc(%ebp), %eax
mov    %eax, 0xfffffeec(%ebp)
mov    0x8(%ebp), %eax
add    $0x20, %eax
```

18. 假设以下 C 语言函数 `compare_str_len()` 用来判断两个字符串的长度, 当字符串 `str1` 的长度大于 `str2` 的长度时函数返回值为 1, 否则为 0。

```
1 int compare_str_len(char *str1, char *str2)
2 {
3     return strlen(str1) - strlen(str2) > 0;
4 }
```

已知 C 标准库函数 `strlen()` 原型声明为 “`size_t strlen(const char *s);`”, 其中, `size_t` 定义为 `unsigned int` 类型。请问: 函数 `compare_str_len()` 在什么情况下返回结果不正确? 为什么? 为使函数正确返回结果, 应如何修改代码?

19. 考虑以下 C 程序代码:

```
1 int func1(unsigned word)
2 {
3     return (int) ((word << 24) >> 24);
4 }
5
6 int func2(unsigned word)
7 {
8     return (int) word << 24 >> 24;
9 }
```


假设在一个 32 位机器上执行这些函数，该机器使用二进制补码表示带符号整数。无符号整数采用逻辑移位，带符号整数采用算术移位。请填写表 2.14，并说明函数 func1() 和 func2() 的功能。

表 2.14 题 19 表

w		func1(w)		func2(w)	
机器数	值	机器数	值	机器数	值
	127				
	128				
	255				
	256				

20. 填写表 2.15，注意对比无符号整数和带符号整数的乘法结果及截断操作前后的结果。

表 2.15 题 20 表

模式	x		y		x × y (截断前)		x × y (截断后)	
	机器数	值	机器数	值	机器数	值	机器数	值
无符号	1101		0101					
带符号	1101		0101					
无符号	0011		1111					
带符号	0011		1111					
无符号	1111		1111					
带符号	1111		1111					

21. 以下 C 函数 arith() 是直接 C 语言写的，而 optarith() 是对 arith() 函数以某个确定的 M 和 N 编译生成的机器代码反编译生成的。根据 optarith() 推断函数 arith() 中 M 和 N 的值各是多少，并推断编译器对 arith() 函数做了哪些优化。

```
1 #define M
2 #define N
3 int arith(int x, int y)
4 {
5     int result = 0 ;
6     result = x*M + y/N;
7     return result;
8 }
9
10 int optarith ( int x, int y)
11 {
12     int t = x;
13     x <<= 4;
14     x -= t;
15     if ( y < 0 ) y += 3;
16     y >>= 2;
17     return x+y;
18 }
```

22. 下列几种情况所能表示的数的范围是什么？

- (1) 16 位无符号整数
- (2) 16 位原码定点小数

- | | | |
|-----|-------|-------------|
| 数符 | 阶码 | 尾数 |
| 1 位 | 8 位移码 | 7 位原码小数数值部分 |

- +1.75, +19, -1/8, 258**

- 表 2.16 题 26 表

表 2.17 题 29 表

[illegible]

30. 在字长为 32 位的计算机上, 有一个 C 函数原型声明为 “int ch_mul_overflow(int x, int y);”, 该函数用于对两个 int 型变量 x 和 y 的乘积判断是否溢出, 若溢出则返回 1, 否则返回 0。请使用 64 位整型 long long 来编写该函数。

31. 对于 2.6.5 节中例 2.31 存在的整数溢出漏洞, 如果将其中的第 5 行改为以下两个语句:

```
unsigned long long arraysize=count*(unsigned long long)sizeof(int);
int *myarray = (int *) malloc(arraysize);
```

已知 C 标准库函数 malloc() 的原型声明为 “void *malloc(size_t size);”, 其中, size_t 定义为 unsigned int 类型, 则上述改动能否消除整数溢出漏洞? 若能则说明理由; 若不能则给出修改方案。

32. 已知一次整数加法、一次整数减法和一次移位操作都只需一个时钟周期, 一次整数乘法操作需要 10 个时钟周期。若 x 为一个整型变量, 现要计算 $55 * x$, 请给出一种计算表达式, 使所用时钟周期数最少。

33. 假设 x 为一个 int 型变量, 请给出一个用来计算 $x/32$ 的值的函数 div32。要求不能使用除法、乘法、模运算、比较运算、循环语句和条件语句, 可以使用右移、加法以及任何按位运算。

34. 无符号整数变量 ux 和 uy 的声明和初始化如下:

```
unsigned ux=x;
unsigned uy=y;
```

若 sizeof(int)=4, 则对于任意 int 型变量 x 和 y, 判断以下关系表达式是否永真。若永真则给出证明; 若不永真, 则给出结果为假时 x 和 y 的取值。

- | | |
|---|---|
| (1) $(x * x) \geq 0$ | (2) $(x - 1 < 0) \parallel x > 0$ |
| (3) $x < 0 \parallel -x \leq 0$ | (4) $x > 0 \parallel -x \geq 0$ |
| (5) $x \& 0xf \neq 15 \parallel (x \ll 28) < 0$ | (6) $x > y == (-x < -y)$ |
| (7) $-x + -y == -(x + y)$ | (8) $(int) (ux - uy) == -(y - x)$ |
| (9) $((x \gg 2) \ll 2) \leq x$ | (10) $x * 4 + y * 8 == (x \ll 2) + (y \ll 3)$ |
| (11) $x / 4 + y / 8 == (x \gg 2) + (y \gg 3)$ | (12) $x * y == ux * uy$ |
| (13) $x + y == ux + uy$ | (14) $x * -y + ux * uy == -x$ |

35. 变量 dx、dy 和 dz 的声明和初始化如下:

```
double dx = (double) x;
double dy = (double) y;
double dz = (double) z;
```

若 float 和 double 分别采用 IEEE 754 单精度和双精度浮点数格式, sizeof(int)=4, 则对于任意 int 型变量 x、y 和 z, 判断以下关系表达式是否永真。若永真则给出证明; 若不永真则给出结果为假时 x、y 和 z 的取值。

- | | |
|------------------------------------|--|
| (1) $dx * dx \geq 0$ | (2) $(double)(float) x == dx$ |
| (3) $dx + dy == (double) (x + y)$ | (4) $(dx + dy) + dz == dx + (dy + dz)$ |
| (5) $dx * dy * dz == dz * dy * dx$ | (6) $dx / dx == dy / dy$ |

36. 在 IEEE 754 浮点数运算中, 当结果的尾数出现什么形式时需要进行左规, 什么形式时需要进行右规? 如何进行左规, 如何进行右规?

37. 在 IEEE 754 浮点数运算中, 如何判断浮点运算的结果是否溢出?

38. 分别给出不能精确用 IEEE 754 单精度和双精度格式表示的最小正整数。

39. 采用 IEEE 754 单精度浮点数格式计算下列表达式的值。

(1) $0.75 + (-65.25)$ (2) $0.75 - (-65.25)$

40. 以下是函数 `fpower2()` 的 C 程序，用于计算 2^x 的浮点数表示，其中调用了函数 `u2f()`。`u2f()` 用于将一个无符号整数表示的 0/1 序列作为 float 类型返回。请填写 `fpower2()` 函数中的空白部分，以使其能正确计算结果。

```
1 float fpower2(int x) {
2
3     unsigned exp, frac, u;
4
5     if (x < _____) { /* 值太小，返回 0.0 */
6         exp = _____;
7         frac = _____;
8     } else if (x < _____) { /* 返回非规格化结果 */
9         exp = _____;
10        frac = _____;
11    } else if (x < _____) { /* 返回规格化结果 */
12        exp = _____;
13        frac = _____;
14    } else { /* 值太大，返回 +∞ */
15        exp = _____;
16        frac = _____;
17    }
18    u = exp << 23 | frac;
19    return u2f(u);
20 }
```

41. 以下是一组关于浮点数按位级进行运算的编程题目，其中用到一个数据类型 `float_bits`，它被定义为 `unsigned int` 类型。以下程序代码必须采用 IEEE 754 标准规定的运算规则，例如，舍入应采用就近舍入到偶数的方式。此外，代码中不能使用任何浮点数类型、浮点数运算和浮点常数，只能使用 `float_bits` 类型；不能使用任何复合数据类型，如数组、结构体和联合等；可以使用无符号整数或带符号整数的数据类型、常数和运算。要求编程实现以下功能并进行正确性测试。

(1) 计算变量 f 的绝对值 $|f|$ 。若 f 为 NaN，则返回 f ，否则返回 $|f|$ 。函数原型如下：

```
float_bits float_abs(float_bits f);
```

(2) 计算变量 f 的负数 $-f$ 。若 f 为 NaN，则返回 f ，否则返回 $-f$ 。函数原型如下：

```
float_bits float_neg(float_bits f);
```

(3) 计算 $0.5 * f$ 。若 f 为 NaN，则返回 f ，否则返回 $0.5 * f$ 。函数原型如下：

```
float_bits float_half(float_bits f);
```

(4) 计算 $2.0 * f$ 。若 f 为 NaN，则返回 f ，否则返回 $2.0 * f$ 。函数原型如下：

```
float_bits float_twice(float_bits f);
```


(5) 将 int 型变量 i 的位序列转换为 float 型位序列。函数原型如下:

```
float_bits float_i2f(int i);
```

(6) 将变量 f 的位序列转换为 int 型位序列。若 f 为非规格化数, 则返回值为 0; 若 f 是 NaN 或 $\pm \infty$ 或超出 int 型数可表示范围, 则返回值为 0x8000 0000; 若 f 带小数部分, 则考虑舍入。函数原型如下:

```
int float_f2i(float_bits f);
```